englishurlseen = last visited:

main.bib

NTNU
Norwegian University of
Science and Technology

Practical Anonymous Communication with Homomorphic Encryption

Angelo, Imre

Submission date: April 2026
Main supervisor: Associate Professor Park, Jeongeun, NTNU

Norwegian University of Science and Technology
Department of Information Security and Communication Technology

Problem description

Many anonymous communication systems have been proposed and implemented with the goal of allowing users to exchange messages over a network without revealing who is communicating with whom. Most of these designs rely on the *threshold model* to provide anonymity, wherein some components of their infrastructure must be behaving honestly. Systems with stronger anonymity guarantees generally suffer from poor scalability or are impractical to instantiate.

This thesis examines a new construction called a (Somewhat) Practical Anonymous Router (sPAR), which addresses the trust and practicality concerns of existing systems. The construction relies only on well-established Fully Homomorphic Encryption (FHE) schemes, making it realistically implementable. While sPAR presents a promising new avenue for constructing anonymous communication systems, it remains a theoretical construction. A concrete implementation and a deeper investigation is therefore needed to evaluate its practical performance and viability in larger networks.

In an effort to investigate sPAR's viability, this thesis aims to implement the scheme and conduct a structured evaluation of its performance with respect to the number of participants in the system.

Research Questions:

- What is the computational cost of sPAR, and how does it compare to the theoretical complexity described in the original paper?
- How does the performance of sPAR scale with the number of participants, and at what point does it become impractical?

Approved: 2026-04-01 – Associate Professor Park, Jeongeun, NTNU (Main supervisor)

Abstract

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

And after the second paragraph follows the third paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

?abstractname?

After this fourth paragraph, we start a new paragraph sequence. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Preface

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Contents

List of Figures

List of Tables

Chapter 1
Background

Anonymous communication aims to enable users to exchange messages over a network without revealing who is communicating with whom, thereby protecting both sender and receiver identities. Many such systems have been proposed and implemented over the years, such as Tor, I2P, and mixnets. Most of these designs depend on the *threshold model* to provide anonymity, requiring at least some components of their infrastructure to behave honestly. In practice, this assumption is increasingly strained by large-scale surveillance, correlation attacks, and the growing centralization of internet infrastructure [**?**].

Systems with stronger anonymity guarantees based on Multi-Party Communication (MPC) exist. However, these suffer from poor scalability due to their high interactivity, making them impractical for large-scale networks. A Dining Cryptographers network (DC-net), for example, can provide anonymity even if there are no trusted components in the network infrastructure. Doing so requires every participant to interact with every other participant, for every round of messages sent.

Recently, a new line of research has proposed fundamentally different designs, aiming to offer strong anonymity even in the presence of compromised or adversarial components in the network actively attempting to break anonymity. The first such design was the Non-Interactive Anonymous Router (NIAR) [**?**]. This paper proposed using Multi-Client Functional Encryption (MCFE) to allow a single, untrusted router to shuffle or permute messages from a set of clients *obliviously*; without learning anything about the permutation. Thereby making it infeasible for the router to link any of the messages to its sender.

NIAR was a breakthrough in anonymous communication, not only showing the theoretical possibility of designing anonymous routers, but also that such routers can be *non-interactive*: clients only send one message to the router and do not need to perform any additional interaction to achieve anonymity. Using a centralized (anonymous) router in this way yields similar benefits to existing MPC-based systems,

without needing any interactions between clients. However, there are a number of problems that makes NIAR impractical, if not impossible, to implement in practice.

1. The cryptographic components required by NIAR are extremely computationally expensive or not known to be implementable in practice.
2. Achieving anonymity for recipients assumes the existence of a very strong primitive called *indistinguishability obfuscation with sub-exponential security*. The practicality of such a construction is unknown.
3. Reusing the same setup across multiple rounds makes messages from the same sender linkable, unless the expensive setup phase is rerun for every round

- Anonymous communication
- Current solutions
- Problems with current solutions
- FHE as a new approach
- sPAR
- RSA is multiplicatively homomorphic, but (follow-up) paper theorized a *fully* homomorphic scheme may be possible [**?**, **?**]

Chapter 2
Related Works

This section details RGSW (External/Internal products), HomExpand, and the sPAR protocol itself.

2.1 TFHE (RGSW)

The GSW ciphertext defined in [**?**], now in the ring R_q. It works by changing bases. Given a base B and decomposition parameter ℓ, such that the message μ can be expressed with ℓ digits in base B, define the *gadget vector* as $g = (1, B, \ldots, B^{\ell-1})^\intercal \bmod t$ and define the *gadget matrix* G as (2.1).

$$G = I_2 \otimes g = \begin{bmatrix} g & 0 \\ 0 & g \end{bmatrix} \tag{2.1}$$

Then an Ring-GSW (RGSW) ciphertext C encrypting a message $\mu \in [0, N)$ is defined by (2.2), where each row of Z is an Ring Learning With Errors (RLWE) encryption of 0 under the secret key s (effectively rows of public keys), μ is the message and G is the gadget matrix (duh).

$$C = Z + \mu \cdot G \tag{2.2}$$

While (2.2) is sufficient to construct $\mathsf{RGSW}_s(\mu)$, we can speed up the process by directly encrypting each row. (Note RGSW can be viewed as a flat vector of 2ℓ RLWE ciphertexts where the first ℓ modify a and the last ℓ modify b).

Maybe use $\mathsf{RLWE} = (b, a)$ like OpenFHE uses in e.g. →`GetElements()`? In which case the top and bottom rows swap place.

Implementation details below, but ultimately not used

Corollary 2.1. *For an RGSW ciphertext C encrypting a message μ under s, the $(\ell+i)$-th row $(0 \leq i < \ell)$ is equivalent to:*

$$C_{\ell+i} = \mathsf{RLWE}_s(\mu B^i/\Delta) \tag{2.3}$$

Proof. Follows from the definitions (2.2) and (2.1).

$$\begin{aligned} Z_i + \mu \cdot G_i &= \mathsf{RLWE}_s(0) + \mu \cdot (0, g_i) \\ &= (a, a \cdot s + \epsilon) + (0, \mu \cdot B^i) \\ &= (a, a \cdot s + \mu B^i + \epsilon) \\ &= \mathsf{RLWE}_s(\mu B^i/\Delta) \end{aligned}$$

□

The same approach shows the first ℓ rows can be constructed as an RLWE encryption $c_i = \mathsf{RLWE}_s(-s\mu B^i/\Delta)$ for $0 \leq i < \ell$ (Theorem 2.2). However, calculating $s \cdot \mu$ is not trivial in practice, and it's faster to simply modifying the a component of a fresh $\mathsf{RLWE}(0)$ encryption. Equation 2.1 creates $\mathsf{RGSW}_s(\mu)$ in $\mathcal{O}(\ell)$ steps (but can only be performed on the client-side as it requires s).

[ht!] Client-side RGSW generation [1] **Input:** Polynomial $\mu = \sum_j b_j X^j, \quad b_j \in [0, N)$ **Input:** Decomposition parameter ℓ**Output:** $\mathsf{RGSW}_s(\mu)$ Construct a $2\ell \times 1$ vector C $i = 0,\ i < \ell$ Let $\hat{\mu} = \mu B^i/\Delta = \frac{B^i}{\Delta}(\sum_j b_j X^j)$ note: coeff. $B^i \cdot b_j \bmod q$ $\mathbf{C}_i = \mathsf{RLWE}_s(\hat{\mu})$ $\mathbf{C}_{i+\ell} = \mathsf{RLWE}_s(0) + (0, \hat{\mu})$ $\{\mathbf{C}_k\}_{k=0}^{2\ell-1}$

Note: we focus on BGV, where $\Delta = 1$, which simplifies the algorithm even more.

2.1.1 Parameter Choice

The choice of basis B and decomposition parameter ℓ is one and the same. For a given basis, the message polynomial μ is rewritten into $\log_B(\mu)$ base-B digits. Therefore $\ell \geq \log_B(\mu)$, otherwise the RGSW does not encode all the digits, and $\ell \leq \log_B(\mu)$ because more B-digits would always be 0. The value of ℓ is then squeezed and must equal the number of base-B digits that encodes all μ values allowed by the scheme.

2.1.2 External Product

First defined in [**?**]? The external product (2.5) is defined between an RLWE and a RGSW ciphertext, and outputs an RLWE ciphertext without needing to use homomorphic multiplication (and thus relinearization). The external product uses a *decomposition matrix* G^{-1} that has the *gadget property* (2.4), which effectively "undoes" the effect of the gadget matrix G (with some error $\leq B/2$, assuming x has binary elements).

$$G^{-1}(x) \cdot G = x + e \pmod{q} \tag{2.4}$$

$$x \boxdot Y = G^{-1}(x) \cdot Y \tag{2.5}$$

2.1.3 BGV-RNS Optimizations

Some key observations allow us to much more efficiently port the RGSW operations to BGV-RNS.

1. RNS ciphertexts are decomposed by Chinese Remainder Theorem (CRT) using $\{q_i\} \; \forall \; i$ as a basis.
2. If we choose $g = (q_0, q_1, \ldots, q_\ell)$ as the gadget vector, then we get gadget de/composition for "free" as it is already baked into the RNS structure

2.1.4 Internal Product

RGSW $\times$ RGSW

$$A \boxtimes B = \begin{bmatrix} A \boxdot b_0 \\ A \boxdot b_1 \\ \vdots \\ A \boxdot b_\ell \end{bmatrix} \tag{2.6}$$

2.1.5 RLWE-to-RGSW Expansion from ORAM

As established in section 2.1, RGSW ciphertexts are enticing because the external product allows us to perform homomorphic multiplication without relinearization. However, the size of RGSW makes the communication costs very high in a communication protocol. We wish to strike a balance between the cheap communication cost of packed RLWE and the fast multiplication of RGSW. The solution is to send RLWE ciphertexts from the client, and *expand* them homomorphically on the server.

Solution from chillotti:onionring:2019. **HomExpand:** Expands an RLWE ciphertext into an RGSW ciphertext.

This expansion is described in Algorithm 4 of [**?**], which builds on Algorithms 3 and 8 from the same paper. The algorithms are listed below (in slightly different notation) as subsection 2.1.5, subsection 2.1.5, and **??**, respectively.

for each S[FOR]ForEach[1] 1

[ht!] HomExpand [1] **Input:** $\mathbf{c}_k = \text{RLWE}(\sum_{i=0}^{n-1} \frac{b_i}{nB^{k+1}})$, $b_i \in \{0,1\}$, $0 \leq k < \ell$ **Input:** $A = \text{RGSW}(-s)$ **Output:** $\mathbf{C}_i = \text{RGSW}(b_i)$, $0 \leq i < n$ k $\mathbf{c'}_k := \mathsf{expandRlwe}(c_k)$ $i = 0$, $i < n$ Initialize $\mathbf{C}_i$ as an empty $2\ell \times 2$ matrix of polynomials $k = 0; k < \ell$ $\mathbf{C}_i[k] = A \boxdot \mathbf{c'}_k[i]$ $\mathbf{C}_i[k+\ell] = \mathbf{c'}_k[i]$ $\{\mathbf{C}_j\}_{j=0}^{n-1}$

subsection 2.1.5 takes as input ℓ RLWE ciphertexts, which in sPAR correspond to the ℓ encrypted bits of the index a_j. It also takes an RGSW ciphertext of the encrypted secret key (which key, automorphism? Sent by the client or generated by server?).

It begins by running a subroutine expandRlwe (subsection 2.1.5) which converts each of the ℓ RLWE ciphertexts into a matrix, like a *partial* RGSW. Then these partial RGSW's are combined into a single (complete) RGSW.

[ht!] expandRlwe [1] **Input:** $\mathbf{c} = \text{RLWE}(\sum_{i=0}^{n-1} b_i X^i)$, $b_i \in \{0,1\}$ **Output:** $\mathbf{c}_i = \text{RLWE}(n \cdot b_i)$, $0 \leq i < n$ $\mathbf{c}_0 := \mathbf{c}$ $i = 0$, $i \leq \log n$ $k = n/2^{i-1} + 1$ $b = 0; b < 2^i$ $\mathbf{c}_{2b} = c_b + \text{Subs}(c_b, k)$ $\mathbf{c}_{2b+1} = c_b - \text{Subs}(c_b, k)$ $\{\mathbf{c}_j\}_{j=0}^{n-1}$

This subroutine converts an RLWE ciphertext with ℓ slots into ℓ ciphertexts, each encoding a unique slot of the input. Subs is a function that... (EvalAutomorphism!)

Their method is fast for TFHE, but for our use — case where $n << N$ — it's faster to use the method described in [**?**] to repeatedly "rotate" the ciphertext ℓ times as in subsection 2.1.5. The FastRotate operation is supported by the OpenFHE as EvalFastRotate.

Note that the components being scaled by n is a side-effect of their implementation, and not required when using FastRotate, so the output is a list of $c_i = \mathsf{RLWE}(b_i)$.

[ht!] expandRlwe [1] **Input:** $\mathbf{c} = \mathrm{RLWE}(\sum_{i=0}^{n-1} b_i X^i),\ b_i \in \{0,1\}$ **Output:** $\mathbf{c}_i = \mathrm{RLWE}(b_i),\ 0 \leq i < n\ \mathbf{c}_0 := \mathbf{c}\ i = 0,\ i \leq \log n$ do something... $\{\mathbf{c}_j\}_{j=0}^{n-1}$

2.1.6 Homomorphic Field Trace

New paper!

2.2 Implementation Details

2.2.1 RGSW

In practice, it would be convenient to construct each row directly, rather than first constructing both matrices in (2.2) and combining them.

Recall that an RLWE ciphertext is a vector $\mathsf{RLWE}(\mu) = (a, b)$ where $b = a \cdot s + \mu + \epsilon$ *(since $\Delta = 1$ in BGV)*. We can view RGSW as a flat vector of 2ℓ RLWE ciphertexts, or a $2\ell \times 2$ matrix with an a column and b column. In the latter representation, the first ℓ rows of C are expressed as:

$$C_i = (a,\ b_{\mu=0}) + \mu \cdot (1/B^i,\ 0) = (a + \mu/B^i,\ a \cdot s + \epsilon)$$

As each row of an RGSW ciphertext is a valid RLWE ciphertext, we can find a new message $\hat{\mu}_i$ such that $\mathsf{RLWE}(\hat{\mu}_i) = (\hat{a}, \hat{a} \cdot s + \Delta \cdot \hat{\mu}_i + \epsilon) = C_i$. Starting with the top half, let $\hat{a} = a + \mu \cdot B^i$ so that the first component matches.

$$\begin{aligned} b &= (a + \mu \cdot B^i) \cdot s + \Delta \cdot \hat{\mu}_i + \epsilon && (2.7)\\ &= a \cdot s + s \cdot \mu \cdot B^i + \Delta \cdot \hat{\mu}_i + \epsilon && (2.8)\\ &= a \cdot s + (s\mu B^i + \Delta\hat{\mu}_i) + \epsilon && (2.9) \end{aligned}$$

The middle term must equal 0 to match C_i, which leads us to an expression (2.12) for $\hat{\mu}_i$.

$$\begin{aligned} 0 &= s \cdot \mu \cdot B^i + \Delta \cdot \hat{\mu}_i && (2.10)\\ \hat{\mu}_i &= -s \cdot \mu \cdot B^i \cdot 1/\Delta && (2.11)\\ \hat{\mu}_i &= -s \cdot \mu \cdot B^i && (2.12) \end{aligned}$$

$\therefore$ we can construct row i (0-indexed) of $\mathsf{RGSW}_s(\mu)$ as $\mathsf{RLWE}_s(-s \cdot \mu \cdot B^i)$ for $0 \le i < \ell$.Further, the RLWE plaintext μ in mod t so we can also use $\mathsf{RLWE}_s(t - s\mu B^i)$.

$$\begin{aligned} \mathsf{RLWE}_s(-s\mu B^i) &= (a, a \cdot s - s\mu B^i + \epsilon) \pmod t\\ \mathsf{RLWE}_s(t - s\mu B^i) &= (a, a \cdot s + (t - s\mu B^i) + \epsilon) \pmod t\\ \mathsf{RLWE}_s(-s\mu B^i) &\equiv \mathsf{RLWE}_s(t - s\mu B^i) \pmod t \end{aligned}$$

For the bottom ℓ rows, we can skip Z entirely since, as **??** says:

$$\mathsf{RLWE}_s(0) + (0, \mu B^i) = (a,\ a \cdot s + \mu B^i + \epsilon) = \mathsf{RLWE}_s(\mu B^i)$$

Appendix?

Lemma 2.2. *The i-th row C_i of $\mathsf{RGSW}_s(\mu)$ is:*

$$C_i = \mathsf{RLWE}_s(-s\mu B^i/\Delta) \tag{2.13}$$

Proof. Let $\hat{a} = a + \mu B^i$, and consider $\mathsf{RLWE}_s(\hat{\mu})$.

$$\begin{aligned}\mathsf{RLWE}(\hat{\mu}) &= (\hat{a}, \hat{a} \cdot s + \Delta\hat{\mu} + \epsilon) \\ &= (a + \mu B^i, (a + \mu B^i) \cdot s + \Delta\hat{\mu} + \epsilon) \\ &= (a + \mu B^i, a \cdot s + \mu B^i \cdot s + \Delta\hat{\mu} + \epsilon)\end{aligned}$$

Assume $C_i = \mathsf{RLWE}(\hat{\mu})$, then:

$$\begin{aligned}s\mu B^i + \Delta\hat{\mu} &= 0 \\ \therefore \hat{\mu} &= -s\mu B^i/\Delta\end{aligned}$$

□

RNS optimizations

OpenFHE is built on a Residue Number System (RNS) where each polynomial is represented as residues of smaller (typically 48 bits for AVX512 or 60 bits by default) primes. The polynomials are equivalent to their non-RNS counter-parts by the Chinese Remainder Theorem.

The base should be 60 (or 48) aka. `NativeSize` (?) in OpenFHE, then the gadget decomposition param becomes (lower bound)...

2.2.2 External Product for BGV-RNS

Expand the condition (**??**):

$$G^{-1}(x) \cdot \begin{pmatrix} g & 0 \\ 0 & g \end{pmatrix} = (a,\ a \cdot s + \mu) + \epsilon \pmod q \tag{2.14}$$

Due to the 0's, the top ℓ rows correspond to the public component a, while the bottom ℓ rows correspond to the encrypted message b. So G^{-1} is a vector with 2ℓ components, which we can split into two distinct vectors denoted G^{-1}_{top} and G^{-1}_{bot} for now.

$$G^{-1}_{\text{top}}(x) \cdot g = \sum_{i=1}^{l} u_i/B^i \approx a + \epsilon \pmod q \tag{2.15}$$

$$G^{-1}_{\text{bot}}(x) \cdot g = \sum_{i=1}^{l} v_i/B^i \approx a \cdot s + \mu_x + \epsilon \pmod q \tag{2.16}$$

$$G^{-1}_{\text{RNS}}(a)_j = a \bmod \alpha_j \in R_{\alpha_j}, \qquad g_j = \hat{\alpha}_j = \frac{q}{\alpha_j} \tag{2.17}$$

$$G^{-1}_{\text{RNS}}(a) \cdot g = \sum_{j=0}^{\ell-1} (a \bmod \alpha_j) \cdot \hat{\alpha}_j \equiv a \pmod q \tag{2.18}$$

So in the code, we consider $G^{-1}(x)$ as two separate vectors/polynomials $a \cdot g^{-1}$ and $b \cdot g^{-1}$ and then calculate the external product (2.5) as:

$$x \boxdot Y = (a, b) \tag{2.19}$$

$$\text{where } a = (x_1 \cdot g^{-1}) \cdot Y \tag{2.20}$$

$$b = (x_2 \cdot g^{-1}) \cdot Y \tag{2.21}$$

$$a = \sum_{i=0}^{n} c_i \cdot x^i \quad \Longrightarrow \quad a \cdot b = \sum \tag{2.22}$$

2.3 Residue Number System

The lattice-based schemes BFV, BGV, and CKKS all operate in a polynomial ring $R_q = \mathbb{Z}_q[X]/(X^N + 1)$ where q is the *ciphertext modulus*. To support a multiplicative depth L, q must be on the order of 2^{60L} bits or more — far larger than a 64-bit machine word. Doing arithmetic on such large numbers natively would require expensive multi-precision routines for every coefficient operation.

The RNS sidesteps this by representing each coefficient as a tuple of small residues, then performing arithmetic component-wise. The reconstruction is given by the CRT.

2.3.1 The Chinese Remainder Theorem

Theorem 2.3. CRT *Let $q_0, q_1, \ldots, q_{k-1}$ be pairwise coprime moduli with $q = \prod_i q_i$. The map*

$$\phi : \mathbb{Z}_q \to \prod_{i=0}^{k-1} \mathbb{Z}_{q_i}, \qquad x \mapsto ([x]_{q_0}, [x]_{q_1}, \ldots, [x]_{q_{k-1}}) \tag{2.23}$$

is a ring isomorphism. The inverse is given explicitly by

$$\phi^{-1}(x_0, \ldots, x_{k-1}) = \sum_{i=0}^{k-1} x_i \cdot \hat{q}_i \cdot [\hat{q}_i^{-1}]_{q_i} \pmod{q}, \qquad \hat{q}_i = q/q_i. \tag{2.24}$$

The same map lifts to polynomial rings: $R_q \cong \prod_i R_{q_i}$. The factor q_i is called an *RNS limb* or *tower*, and $\hat{q}_i$ is the *punctured product*.

For OpenFHE, each q_i is a 60-bit prime (or 48-bit on AVX-512 builds) chosen to support the Number Theoretic Transform (NTT) for the ring degree N. With this choice, addition and multiplication of two CRT-represented polynomials reduce to k independent native-word operations per coefficient.

2.3.2 CRT representation in code

We write $\mathsf{CRT}(a) = (a^{(0)}, a^{(1)}, \ldots, a^{(k-1)})$ with $a^{(i)} \in R_{q_i}$. Addition and multiplication are component-wise:

$$\mathsf{CRT}(a + b) = (a^{(i)} + b^{(i)})_i, \qquad \mathsf{CRT}(a \cdot b) = (a^{(i)} \cdot b^{(i)})_i. \tag{2.25}$$

The cost of multiplication of two degree-N polynomials in R_{q_i} is dominated by the convolution. Naively this is $\mathcal{O}(N^2)$, but the NTT reduces it to $\mathcal{O}(N \log N)$.

2.3.3 Double-CRT representation

The NTT is essentially a discrete Fourier transform performed modulo q_i. It maps the polynomial $a^{(i)} \in R_{q_i}$ from *coefficient form* (a list of N coefficients) to *evaluation form* (a list of N values, one per primitive $2N$-th root of unity in $\mathbb{Z}_{q_i}$). In evaluation form, polynomial multiplication becomes coordinate-wise multiplication — a single point-wise loop instead of a convolution.

The *Double-CRT* (DCRT) representation stores each $a^{(i)}$ in evaluation form across all towers. This is OpenFHE's `DCRTPoly` class. Conversion between coefficient and evaluation form is via forward/inverse NTT, costing one $\mathcal{O}(N \log N)$ pass per tower.

See ?? in the appendix for details on the NTT.

2.3.4 The auxiliary modulus and the QP basis

For some operations — in particular, key switching and the gadget-based decomposition we develop in **??** — it is useful to temporarily extend the working modulus from Q to $Q \cdot P$, where $P = \prod_j p_j$ is an auxiliary product of additional primes coprime to Q.

We refer to the basis $\{q_0, \ldots, q_{k-1}\}$ as the *Q-side* and $\{p_0, \ldots, p_{m-1}\}$ as the *P-side*. A polynomial $a \in R_{QP}$ is then represented by $k + m$ towers: the first k in the Q-side, the last m in the P-side. We write $\mathsf{sizeQ} = k$ and $\mathsf{sizeP} = m$.

Two operations between the bases are crucial:

ApproxSwitchCRTBasis ($Q \to QP$). Given $a \in R_Q$ in CRT form, compute the P-side residues by extending the basis. The exact reconstruction would require knowing a as an integer in $[0, Q)$, but in practice we work with a small-norm representative (assuming $\|a\| \ll Q$) and recover the P-side via the formula

$$a^{(p_j)} = \sum_{i=0}^{k-1} a^{(q_i)} \cdot [\hat{q}_i^{-1}]_{q_i} \cdot \hat{q}_i \pmod{p_j} - v \cdot Q \pmod{p_j} \tag{2.26}$$

for some small correction v that the approximate variant simply drops. The error is bounded by $\|a\| \cdot k/2$, which stays well below p_j for reasonable parameters halevi2019improved.

ApproxModDown ($QP \to Q$). The inverse: given $x \in R_{QP}$, compute $\lfloor x/P \rceil \in R_Q$. Concretely, this subtracts the P-side (re-extended into the Q-side) and multiplies by $[P^{-1}]_{q_i}$ in each Q-tower. For BGV, an additional correction maintains correctness modulo the plaintext modulus t.

The crucial identity for what follows is:

Lemma 2.4. ***P*-cancellation** *For any $x \in R_Q$ embedded into R_{QP} as $(x, 0)$ (i.e. Q-side is x, P-side is 0),*

$$\mathsf{ApproxModDown}(P \cdot x) \approx x \pmod{Q}, \tag{2.27}$$

where the approximation hides the basis-extension rounding error introduced by the P-side correction.

Sketch. Multiplication by P on the Q-side is well-defined modulo Q (since P is invertible there). ApproxModDown subtracts the basis-extended P-side and multiplies by $[P^{-1}]_{q_i}$, which inverts the multiplication by P exactly when the P-side is zero — as it is by construction. □

This lemma is the engine driving the V2 RGSW external product: the gadget g_j defined in **??** packages a free factor of P into each row, and ApproxModDown is what removes it on the way out.

2.4 (Somewhat) Practical Anonymous Routing

The main protocol.

2.4.1 HomPlacing

Algorithm 1 shows the idea of Algorithm 2.

- From NIAR: sorting of random numbers = shuffling.
- Evaluate tree, each bit determines if the data is placed left or right. One level per bit means there are 2^n buckets in the end, each one representing a bit string i.e. 000, 001, 010, 011, 100, 101, 110, 111 for 3 bits.
- We can encode the bits into RGSW form.

Homomorphic placing HomPlacing for one slot [1] **Input:** An encrypted value V_r and $\log n$ bits $\{c_0, \ldots, c_{L-1}\}$, where $L = \log n$ **Output:** A vector $\mathbf{l}$ consisting of n values of leaves: $z_0, \ldots, z_{n-1}$, where $\mathbf{l}[i] = z_i$ for $i \in \{0, \ldots, n-1\}$

$\mathbf{l} := \mathbf{0}$ Initialize root: $b_0 = V_r$ Initialize the node values: $b_i := 0$ for $i \in \{1, \ldots, 2n-2\}$

$i \leftarrow 0, \ldots, L-1$ $j \leftarrow 0, \ldots, 2^i - 1$ $r := b[2^i - 1 + j]$ $b_{2^{i+1}+2j} := r \boxdot c_i$ $b_{2^{i+1}+2j-1} := r - r \boxdot c_i$ $i \leftarrow 0, \ldots, n-1$ $z_i := b_{2^{l+1}-1+i}$ $\mathbf{l}[i] = z_i$ $\mathbf{l}$

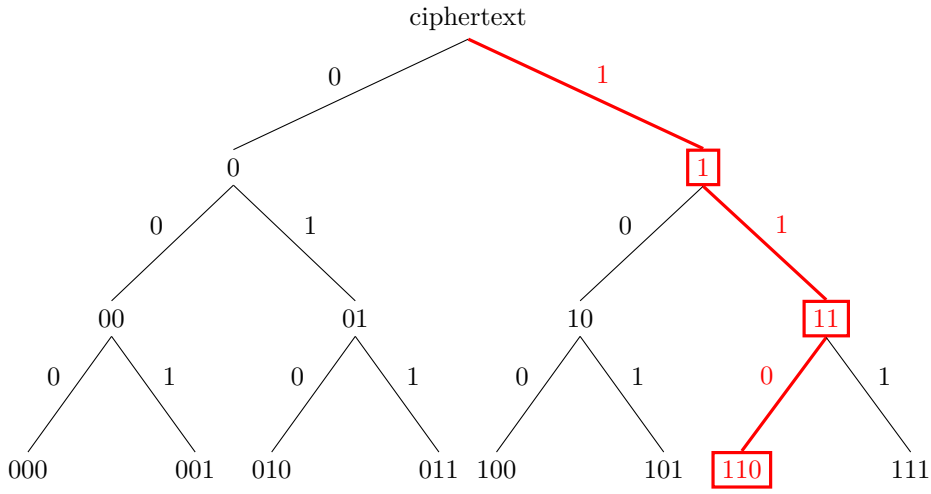

Figure 2.1: HomPlacing tree with a 3-bit index a. At level i, the i-th MSB of the index a determines homomorphically wether the ciphertext is placed in the left (0) or right (1) bucket. This process is repeated $\log n = 3$ times until the ciphertext is placed in the a-th bucket, without the placer ever knowing which bucket that is. The highlighted path shows the route taken for the index $a = 6 = [110]_2$: the MSB places the ciphertext in the right bucket, and so on. Again, the server doesn't know a, only the *encryption* of the bits $[a]_2$.

Listing 1: Algorithm 1 from sPAR implemented in C++ with OpenFHE

2.5 Multi-Party Homomorphic Encryption

TODO

Chapter 3
Methodology

3.1 Variables

- **Plaintext modulus** t - Coefficients of plaintext polynomial are taken mod t
- **Ciphertext modulus** Q - Decomposed into $Q = q_0 q_1 \dots q_L$

3.2 Notes

In Brakerski's scheme, similar to previous FHE schemes based on LWE, the ciphertext $\vec{c}$ and secret key $\vec{s}$ are n-dimensional vectors whose dot product $\langle \vec{c}, \vec{s} \rangle \approx \mu$ equals the message μ, up to some small "error" that is removed by rounding.

3.3 Residue Number System

In RNS, each ciphertext is already decomposed mod $Q_L = q_0 q_1 \dots q_L$. If, then, we select a gadget consisting of $(q_0, q_1, \dots, q_L)$, we get the decomposition "for free".

Chapter 4
Results and Discussion

Chapter 5

Conclusion and Future Work

[title=References]

Appendix A
Algebra

A refresher on relevant basic algebra and number theory. Maybe some can be used in the thesis or as an appendix.

A.1 Rings

A polynomial with coefficients in $\mathbb{F}$ is denoted $\mathbb{F}[X]$. The quotient of a ring $\mathbb{F}[X]/S$ is just $\mathbb{F}[X]$ mod S. Most FHE schemes are based on rings (RLWE, RGSW etc.), and in particular we often see the ring $(\mathbb{Z}/q\mathbb{Z})[X]/(f(X))$ for some integer q and some polynomial $f(X)$. Here $(\mathbb{Z}/q\mathbb{Z})[X]$ denotes a polynomial with coefficients in $\mathbb{Z}_q$ i.e. integers mod q, and since we have $\mathbb{Z}_q[X]/(f(X))$ it means that $f(x) \equiv 0$ in this ring. A natural choice is $f(X) = X^n + 1$ for many of these schemes for some variable n, as this allows all polynomials with degree (up to) n. (And is a cyclotomic polynomial → has additional structure that can be used to optimize performance.)

If $f(X)$ is irreducible over $\mathbb{F}$, then $\mathbb{F}[X]/(f(X))$ is a field. (And $f(X)$ is a maximum ideal of $\mathbb{F}[X]$.)

A factor group G/H, pronounced *G mod H*, is a factor group where H **is the identity element**. In other words, $\mathbb{F}[X]/(f(X))$ has the (additive) identity $f(X)$ aka. $f(X) \equiv 0$ in the ring.

A.2 Cyclotomic Polynomials

$(X^n + 1)$ is a *cyclotomic polynomial*. They are irreducible over $\mathbb{Z}$ as long as $n = 2^m$, $m \in \mathbb{Z}$ (in other words, n is a power of 2) but are typically not irreducible over $\mathbb{Z}_q$.

Definition A.1. For any positive integer n, the n-th roots of unity are the (complex) solutions to the equation $x^n = 1$, and there are n solutions to the equation.

Let $\zeta_n(x) = e^{2\pi i}$. It follows from Euler's formula $e^{2\pi i} = \sin 2\pi + i \cos 2\pi = 1$ that $\zeta_n^k(x) = \{1, e^{2\pi i}, e^{4\pi i}, \ldots, e^{2k\pi i}\}$ for $0 \leq k < n$ are the n-th roots of unity. At least this is the case over $\mathbb{C}$, but we can define the same problem over different fields.

Geometrically, we can interpret the n-th roots of unity as the points that are evenly spread on the unit circle in the complex plane, starting from 1 on the real axis. (The word *cyclotomic* means *circle-dividing.*)

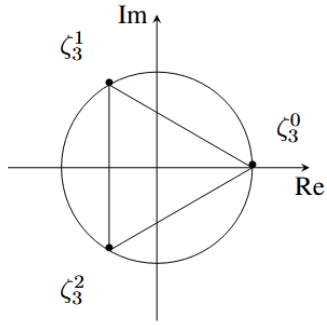

Figure A.1: TODO: Create in tikz or drop

We can define this notion of different fields, and are most interested in defining it over a *finite* $\mathbb{Z}_{p^k}$. As an example, consider $\mathbb{Z}_7 = \{0, 1, 2, 3, 4, 5, 6\}$. The 4-th roots of unity are only 1 and 6, as these are the only values equivalent to 1 when raised to the fourth power. Note that this field has $\phi(n) = \phi(4) = 2$ number of n-th roots for $n = 4$. This is always the case for any finite field and choice of n.

$$1^4 \equiv 1 \mod 7 \tag{A.1}$$

$$6^4 \equiv 1 \mod 7 \tag{A.2}$$

Theorem A.2. *A finite field $\mathbb{Z}_{p^k}$ has a primitive n-th root if and only if $n \mid (p^k - 1)$. In this case, there are $\phi(n)$ roots of unity!*

Definition A.3. An n-th root of unity r is called **primitive** if it is not a d-th root of unity for any integer d smaller than n.

Theorem A.4. *The n-th primitive roots of unity are precisely:*

$$\{\zeta_n^k \,:\, 1 \leq k \leq n-1,\ \gcd(k, n) = 1\} \tag{A.3}$$

Definition A.5. The n-th cyclotomic polynomial $\Phi_n(x)$ has the n-th primitive roots of unity as its roots:

$$\Phi_n(x) = \prod_{\substack{1 \leq k < n \\ \gcd(k,n)=1}} (x - \zeta_n^k) \tag{A.4}$$

Again, we care only about a special class of cyclotomic polynomials that simplify our work. One step in this direction is considering only when n is prime:

$$\Phi_n(x) = x^{n-1} + x^{n-2} + \cdots + 1 = \sum_{t=0}^{n-1} x^t \tag{A.5}$$

Now we consider only finite fields $\mathbb{GF}(p^k)$. Then the cyclotomic polynomials look like:

$$\Phi_n(x) = \Phi_p(x^{n/p}) = \Phi_p(x^{p^{k-1}}) = \sum_{t=0}^{p-1} x^{tp^{k-1}} \tag{A.6}$$

And finally, when $p = 2$ so $n = 2^k$ we get (A.7) where $m = 2^{k-1}$, $n = 2m$,.

$$\Phi_n(x) = x^m + 1 \tag{A.7}$$

Cyclotomic polynomials are monic (implied by the definition) and has $\phi(n)$ linear factors. In addition, $\Phi_n(x)$ divides $x^n - 1$. This implies an important relationship:

$$x^n - 1 = \prod_{d|n} \Phi_d(x) \tag{A.8}$$

The identity (A.8) says that a number is an n-th root of unity iff. it is a d-th primitive root of unity for some natural number d that divides n.

A.3 Galois Group of Cyclotomic Polynomials

Galois theory can be used to answer the question of whether the roots of an arbitrary polynomial f can be expressed in terms of its coefficients. The method for doing so leads to the definition of the *splitting field of a polynomial*, which contains the elementary symmetric polynomials and other polynomials of (subsets of) the roots that can be obtained from them??

Definition A.6. Let f be a polynomial with rational coefficents. The splitting field K of f is the smallest field that contains the roots of f.

K is called the splitting field of f because we can split f into linear factors in K.

Appendix B
Number Theory

Used to speed up calculations on polynomials.

B.1 Chinese Remainder Theorem

Works the exact same way as for integers.

B.2 CRT Polynomial

Used to reduce the size of (the representation of) large polynomials, as q is often much larger than a computer word (32- or more commonly 64-bit words)

B.3 Double CRT Polynomial

Used to perform multiplication on the smaller coefficient-first representation of large polynomials.

B.4 Number Theoretic Transform

The fast fourier transform (FTT) is an algorithm often used to simplify calculations in signals processing etc. by moving a continuous function from the time domain to the frequency domain. The NTT works in a similar fashion; it's essentially a FFT mod q used to speed up multiplication of polynomials.

It is performed on DCRT polynomials.